
Breakout

Advice

Nikola Panayotov

Table of Contents

Table of Contents	1
1 Introduction	2
2 Basis for the recommendation	2
3 Technology comparison	2
4 Proposed solution	2
4.1 Game structure.....	2
4.2 Collision approach.....	2
4.3 Interface and scope	3
4.4 Priority order	3
5 Quality and Optional Requirements	3
6 Design and implementation evidence.....	3
7 Validation and completion	4
8 Use of AI during the project	4
9 Sources	4

1 Introduction

I recommend building the first version of Breakout in Java with LibGDX for Windows. This fits my existing Java experience and leaves the main learning effort for game development. The project should deliver one complete, tested level before adding optional features. This advice uses the scope and requirements defined in Breakout Analysis and provides the proposed solution and global LO2 planning.

2 Basis for the recommendation

The game is a small keyboard-controlled arcade game with a moving ball, paddle, brick grid and four screens. The implementation must coordinate input, drawing and game state while keeping collisions reliable. A successful choice should support these needs without requiring unnecessary learning outside the project goals.

I use a qualitative multi-criteria comparison, as suggested by the Analysis and Advice workshop. Existing language knowledge and support for the required game behavior carry the most weight. Control over the Java class structure also matters because object-oriented programming is a learning goal. The comparison is a project judgment, not a measured performance benchmark.

3 Technology comparison

LibGDX provides the closest fit to my goals because it combines Java with the facilities required for this game. Java2D and Swing remain a valid alternative. Java includes input and sound support, so the issue is the extra work of assembling the game infrastructure, not the absence of those capabilities [4–6]. Unity supports C# scripting [7], but that would add learning beyond my chosen Java focus.

Criterion	Java with LibGDX	Java2D with Swing	Unity with C#
Existing knowledge	Uses my strongest language.	Uses my strongest language.	Requires additional C# learning.
Game infrastructure	Provides lifecycle, graphics, input and audio facilities.	Java provides graphics, events and sound, but I must organize the game loop and integrate them.	Provides an editor and game engine, with C# scripting.
Learning focus	Supports practice with Java classes and game behavior.	Provides low-level practice but adds infrastructure work.	Adds an engine workflow alongside a different language.
Fit for this scope	Strong fit for the small game described in this project.	Feasible, with more manual coordination.	Feasible, but less aligned with the stated Java goal.

4 Proposed solution

4.1 Game structure

Use separate responsibilities for the paddle, ball and bricks, with a game controller or main game class coordinating screens and session state. Keep score, lives and restart behavior consistent through a single reset path. A class structure separating these responsibilities is already proposed. The Design activity should refine that structure and show how the objects interact, rather than repeating the technology comparison.

4.2 Collision approach

Start with an axis-aligned bounding box approach, which treats collision areas as rectangles. Confirm its behavior with a small prototype before building the complete level. Brick contacts need a response

appropriate to the side of impact: always reversing vertical direction can produce incorrect results on side hits. After a contact, move the ball clear of the surface and avoid repeated responses to the same overlap.

Fast movement may allow the ball to cross an object between updates. Test at the intended maximum speed and under slower update conditions. If speed limits are insufficient, evaluate smaller movement steps or swept collision checks. These are proposed mitigations to test, not claims that collision accuracy has already been achieved.

4.3 Interface and scope

Use the start, gameplay, win and game over screens already planned for this game. Keep Space for starting, A/D or arrow keys for movement, and R for restarting from either ending. Show score and lives throughout gameplay. Target Windows only for this version, with one layout and no saved progression or settings screen.

4.4 Priority order

Implement all Must haves first, including both endings and complete restart behavior. Then refine the paddle hit-position angle as S01. Add event sounds as S02 if the core passes validation. Background music is C01 and can be omitted. Multiple levels, extra platforms and the flick effect remain outside the baseline.

5 Quality and Optional Requirements

The table below lists quality and priority (Should/Could) requirements that support the proposed solution above, particularly the priority order (4.4) and the interface and scope (4.3).

ID and priority	Requirement	Acceptance check
Q01 Must	Keep the score, lives, ball, paddle and bricks visible and distinguishable.	Inspect the gameplay screen at the target window size. HUD text is readable and does not obscure active objects.
Q02 Must	Keep repeated sessions stable.	Complete five restarts across win and loss states without a crash, duplicate objects or retained score and lives.
S01 Should	Vary the bounce direction by paddle hit position.	Compare left, center and right hits. The outgoing direction changes consistently and returns the ball into play.
S02 Should	Play effects for paddle hits, brick breaks, life loss and endings.	Trigger each event and check that its effect plays once at the intended moment.
C01 Could	Play background music.	If included, confirm it plays during gameplay without duplicating after restart.

6 Design and implementation evidence

The Design portfolio evidence should include the screen flow, interface wireframes and a class diagram. Each design should support an Analysis requirement. For example, the screen flow should show the start, win, loss and restart transitions, and the gameplay wireframe should locate the score and lives.

The Realization evidence should contain the source code, setup and run instructions, and an explanation of the tools used. Build and test in small increments, starting with a working window, then paddle movement, a ball and one collision. Record meaningful differences between the final implementation and the original design.

The source code is kept in a public Git repository so the evidence can be reviewed and rebuilt: <https://github.com/N1ki2K/breakout>. Because the project uses the Gradle wrapper, no separate Gradle installation is needed:

- Run `gradlew.bat lwjgl3:run` on Windows (or `./gradlew lwjgl3:run` on Linux and macOS) from the repository root to launch the game directly.
- Run `gradlew.bat lwjgl3:jar` (or `./gradlew lwjgl3:jar`) to build a runnable JAR, which is written to `lwjgl3/build/libs`.

7 Validation and completion

Use the requirement identifiers in Breakout Analysis to create a test record with test steps, expected behavior, actual behavior, pass or fail status, and evidence. Record the Windows environment and tested build. Include targeted tests of scoring, life deductions and reset logic where practical, alongside manual tests of movement and collision behavior.

Ask peers to start and play without additional verbal instructions. Record confusion about controls, readability or ball behavior, then document any resulting changes. A teacher review can check the technical approach and portfolio coverage. Completion requires every Must have to pass, no unresolved defect that blocks a full session, and runnable source code with supporting evidence.

8 Use of AI during the project

If I use generative AI, I will use it to explain concepts, review small sections of code and help investigate errors. I will check technical suggestions against documentation and test each change before relying on it. I need to be able to explain the code and the choices in my portfolio. This follows the GenAI workshop's emphasis on learning and checking output, and the Realization workshop's advice to build in small steps.

For any AI-assisted implementation, I will retain relevant prompts or inputs, explain what I changed, and record how I checked the result. These are planned working practices and should be supported by actual evidence as development proceeds.

9 Sources

Technical references consulted 7 September 2026:

#	Reference
[1]	libGDX. The application framework. https://libgdx.com/wiki/app/the-application-framework
[2]	libGDX. A Simple Game. https://libgdx.com/wiki/start/a-simple-game
[3]	libGDX. Input handling. https://libgdx.com/wiki/input/input-handling
[4]	Oracle. Java 2D Graphics. https://docs.oracle.com/javase/tutorial/2d/
[5]	Oracle. Writing Event Listeners. https://docs.oracle.com/javase/tutorial/uiswing/events/
[6]	Oracle. Sound. https://docs.oracle.com/javase/tutorial/sound/index.html
[7]	Unity. Introduction to scripting. https://docs.unity3d.com/Manual/intro-to-scripting.html